1. Introduction

Threat modeling is a systematic process that enables the identification and management of security threats that arise at the early stages of designing any system. In this document, a threat model will be constructed for a purposely weak web application that uses Flask, SQLite, and HTML. The application supports user registration, login, product listing, role escalation to seller, product management, and a review system, alongside an administrative interface. Unlike abstract systems, this application exposes real, code-level vulnerabilities such as unsanitised SQL queries, insecure cookie-based authentication, and a lack of output encoding. Architecture involves the client browser accessing the Flask web service, which interacts directly with the SQLite database. The important trust boundaries lie between the client and the server and between the application and the database, where the danger lies when cookies managed by the client are trusted for authentication purposes. The Data Flow Diagram (DFD) shows the interaction among different processes, such as authentication, managing products, and reviewing. The STRIDE methodology is applied to systematically identify threats across these flows. This way, it becomes possible for us to not only classify threats, but also link them directly with implementation flaws. This will enable us to come up with effective countermeasures and include them in the Software Development Lifecycle (SDLC) process.

Figure 1: Data Flow Diagram (DFD)

2. Attack Surface Analysis

The application contains multiple entry points because it depends on unverified user data and uses unsafe design methods. The most critical entry points are HTTP endpoints such as /login, /register, /add, /edit, /delete, /product/, and /admin. The endpoints handle untrusted input, which they use to construct SQL queries and HTML responses. Authentication represents a major weakness. Unlike secure session management, which relies on server-side sessions, the application instead uses cookies on the client side with the username. The cookie is not signed or verified and can therefore be manipulated by an attacker, enabling him/her to act as an administrator. The database interface layer raises significant risks. Every SQL statement is created via string interpolation; thus, the application becomes highly susceptible to SQL injection attacks from various sources, such as user login, user registration, product creation, and review submission. This brings to light all the data underneath, even passwords kept in plain text. Another point where an attack can take place is via the storage of cross-site scripting (XSS), brought about by the review mechanism. Comments left by users on posts are saved and processed without being filtered first. Any authorized user can easily become a seller without any form of verification, while no rigorous checks for endpoint product modifications exist. These vulnerabilities, when combined with cookie manipulation methods, can lead to full privilege escalation. The absence of any form of rate-limiting, validation checks, and security headers further compromises the system's safety from exploitation attacks.

3. STRIDE Threat Modeling

The STRIDE framework is applied to systematically categorise threats across system components and data flows. Each of these categories presents serious vulnerabilities that are a direct result of the poor implementation of the system. The spoofing threat is high because the system utilizes an unsigned username cookie to authenticate the user’s identity. An attacker is able to assume the identity of anyone, including an administrator, by forging this cookie. Database content, such as product details and user privileges, can change due to injections. The issue of repudiation arises since there are no logs and auditing mechanisms. Actions such as deleting a product or gaining additional privileges cannot be traced back to specific users. Information leakage issues arise from SQL injection, plain-text passwords, and the exposure of users' private information within the admin panel. The sensitive information can be acquired conveniently. The vulnerability to a DOS attack is possible because of the lack of appropriate means of handling requests and queries, which will allow the attacker to send too many requests at once to the program. The elevation of privilege attack can be easily executed because of the improper role management and reliance on cookie-based authentication.

6. Security Requirements

The security measures must directly counter the problems identified in the implementation process. The solution to the problem is to redesign the authentication process using secure session management. The sessions must be stored server-side and must have attributes like HttpOnly, Secure, and SameSite. Password hashing should be done with the use of very secure hash functions like bcrypt. The database connection should adhere to the best practices in using parameterized queries and ORMs in order to avoid any SQL injection attacks. It should have validation to make sure that the user's inputs meet certain criteria. Validation can be done on the basis of data types, data lengths, etc. Encoding is very significant while handling XSS attacks. Access controls should always implement RBAC (role-based access control). For security purposes, authentication and authorization processes need to be done in case of any sensitive operations. Role escalation must be authorized or verified through some process. Data encryption must include both in-transit and at-rest encryption of data through the TLS protocol. CSP headers can be implemented in the application. Operational security controls like rate limiting, logging, and monitoring should be enforced to recognize and mitigate any malicious activity. Logs must be tamper-proof and contain sufficient information to conduct a forensic investigation.

8. Summary

This report demonstrates a comprehensive threat model for a vulnerable web application by directly analysing its implementation. Unlike generic models, the assessment identifies real, exploitable weaknesses such as SQL injection, cookie spoofing, and stored XSS. STRIDE analysis made possible the structured categorization of these threats, whereas the process of risk assessment helped in identifying important weaknesses that needed prompt attention. Security requirements suggested by us are specific to the architecture of the system being built and have been developed keeping in mind addressing the underlying cause of the threat. The approach clearly shows defensive thinking in its implementation and is consistent with industry standards. Generally speaking, the assessment demonstrates that insecure design contributes to the increase of the attack surface in a significant way, stressing once again the need for secure programming, authentication, and threat modeling.